

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И
КОМПЬЮТЕРНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине
«БАЗЫ ДАННЫХ»

Вариант № 302848

Выполнил:

Студент группы Р3107

Добрышкин Владимир Александрович

Преподаватель:

Байрамова Хумай Бахруз Кызы

СОДЕРЖАНИЕ

1. Текст задания	3
2. Запрос №1	4
2.1. Описание запроса	4
2.2. Реализация запроса на SQL	4
2.3. Планы выполнения запроса	4
2.4. Ответы на вопросы, представленные в задании	5
2.4.1. Доказательство эквивалентности планов	5
2.4.2. Выбор оптимального плана выполнения запроса	6
2.4.3. Индексы	6
2.4.4. Результат выполнения EXPLAIN ANALYZE	6
3. Запрос №2	7
3.1. Описание запроса	7
3.2. Реализация запроса на SQL	7
3.3. Планы выполнения запроса	8
3.4. Ответы на вопросы, представленные в задании	9
3.4.1. Выбор оптимального плана выполнения запроса	9
3.4.2. Индексы	9
3.4.3. Результат выполнения EXPLAIN ANALYZE	10
4. Выводы по работе	11

ТЕКСТ ЗАДАНИЯ

Введите вариант:

Внимание! У разных вариантов разный текст задания!

Составить запросы на языке SQL (пункты 1-2).

Для каждого запроса предложить индексы, добавление которых уменьшит время выполнения запроса (указать таблицы/атрибуты, для которых нужно добавить индексы, написать тип индекса; объяснить, почему добавление индекса будет полезным для данного запроса).

Для запросов 1-2 необходимо составить возможные планы выполнения запросов. Планы составляются на основании предположения, что в таблицах отсутствуют индексы. Из составленных планов необходимо выбрать оптимальный и объяснить свой выбор.

Изменятся ли планы при добавлении индекса и как?

Для запросов 1-2 необходимо добавить в отчет вывод команды EXPLAIN ANALYZE [запрос]

Подробные ответы на все вышеперечисленные вопросы должны присутствовать в отчете (планы выполнения запросов должны быть нарисованы, ответы на вопросы - представлены в текстовом виде).

1. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Н_ТИПЫ_ВЕДОМОСТЕЙ, Н_ВЕДОМОСТИ.

Вывести атрибуты: Н_ТИПЫ_ВЕДОМОСТЕЙ.НАИМЕНОВАНИЕ, Н_ВЕДОМОСТИ.ИД.

Фильтры (AND):

a) Н_ТИПЫ_ВЕДОМОСТЕЙ.НАИМЕНОВАНИЕ < Презачет.

b) Н_ВЕДОМОСТИ.ДАТА < 1998-01-05.

Вид соединения: LEFT JOIN.

2. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_ОБУЧЕНИЯ, Н_УЧЕНИКИ.

Вывести атрибуты: Н_ЛЮДИ.ИД, Н_ОБУЧЕНИЯ.НЗК, Н_УЧЕНИКИ.ИД.

Фильтры: (AND)

a) Н_ЛЮДИ.ИМЯ < Александр.

b) Н_ОБУЧЕНИЯ.НЗК < 999080.

c) Н_УЧЕНИКИ.ГРУППА > 3100.

Вид соединения: RIGHT JOIN.

ЗАПРОС №1

ОПИСАНИЕ ЗАПРОСА

Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Н_ТИПЫ_ВЕДОМОСТЕЙ, Н_ВЕДОМОСТИ.

Вывести атрибуты: Н_ТИПЫ_ВЕДОМОСТЕЙ.НАИМЕНОВАНИЕ, Н_ВЕДОМОСТИ.ИД.

Фильтры (AND):

а) Н_ТИПЫ_ВЕДОМОСТЕЙ.НАИМЕНОВАНИЕ < Перезачет.

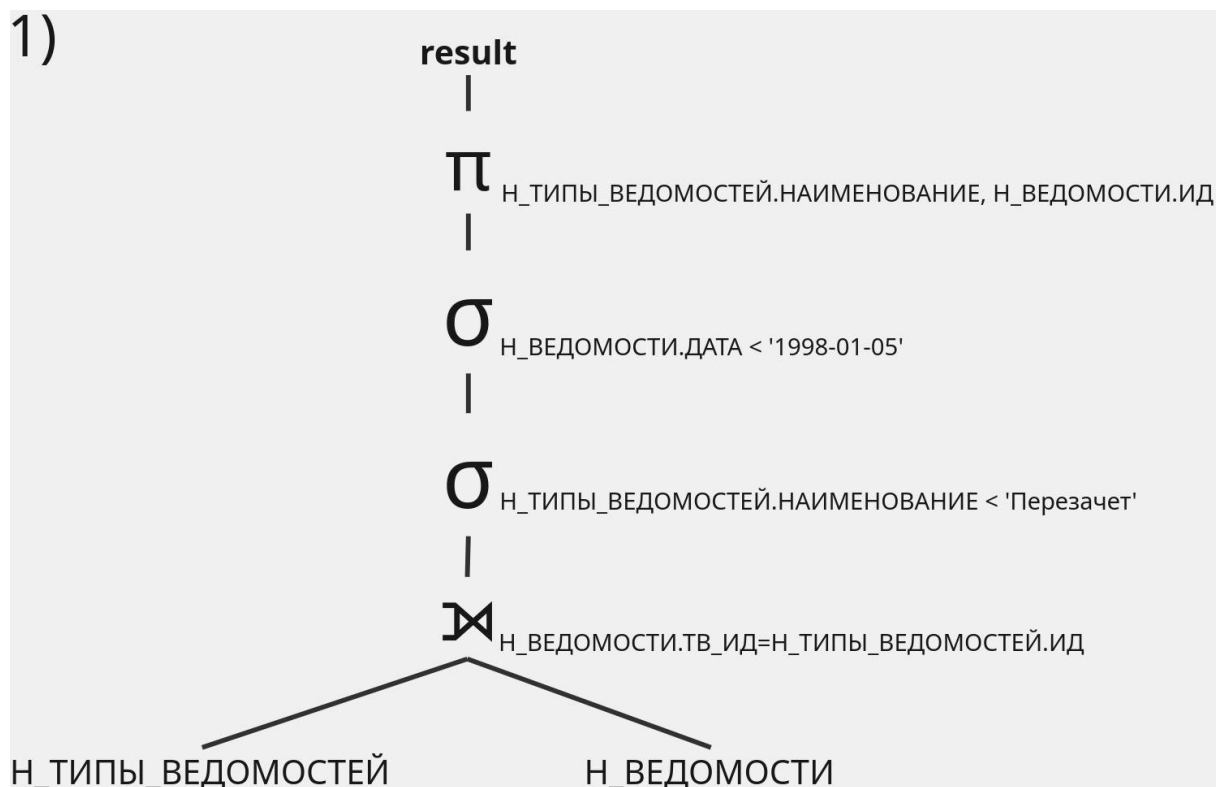
б) Н_ВЕДОМОСТИ.ДАТА < 1998-01-05.

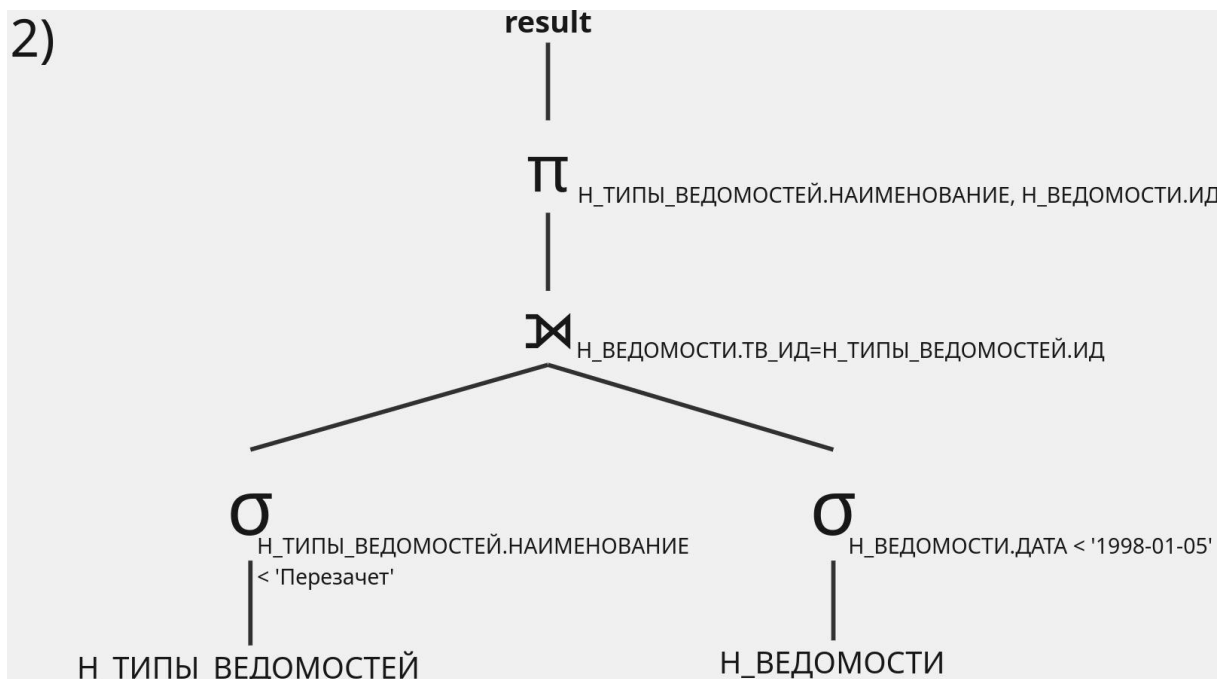
Вид соединения: LEFT JOIN.

РЕАЛИЗАЦИЯ ЗАПРОСА НА SQL

```
select "Н_ТИПЫ_ВЕДОМОСТЕЙ"."НАИМЕНОВАНИЕ", "Н_ВЕДОМОСТИ"."ИД" from
    "Н_ТИПЫ_ВЕДОМОСТЕЙ" left join "Н_ВЕДОМОСТИ" on
    "Н_ВЕДОМОСТИ"."ТВ_ИД" = "Н_ТИПЫ_ВЕДОМОСТЕЙ"."ИД"
where "Н_ТИПЫ_ВЕДОМОСТЕЙ"."НАИМЕНОВАНИЕ" < 'Перезачет' and
    "Н_ВЕДОМОСТИ"."ДАТА" < '1998-01-05';
```

ПЛАНЫ ВЫПОЛНЕНИЯ ЗАПРОСА





ОТВЕТЫ НА ВОПРОСЫ, ПРЕДСТАВЛЕННЫЕ В ЗАДАНИИ

ДОКАЗАТЕЛЬСТВО ЭКВИВАЛЕНТНОСТИ ПЛАНОВ

□ Введём обозначения:

- $Н_ТИПЫ_ВЕДОМОСТЕЙ - R$.
- $Н_ВЕДОМОСТИ - S$.
- $Н_ТИПЫ_ВЕДОМОСТЕЙ.НАИМЕНОВАНИЕ - e_1$.
- $Н_ВЕДОМОСТИ.ИД - d_1$.
- Условие соединения $Н_ВЕДОМОСТИ.ТВ_ИД = Н_ТИПЫ_ВЕДОМОСТЕЙ.ИД - \theta$.
- Предикат $Н_ТИПЫ_ВЕДОМОСТЕЙ.НАИМЕНОВАНИЕ < 'Перезачёт' - \varphi$.
- Предикат $Н_ВЕДОМОСТИ.ДАТА < '1998-01-05' - \Phi$.

Первый план можно представить как $\pi_{e_1, d_1}(\sigma_{\varphi \wedge \Phi}(R \bowtie_{\theta} S))$.

Раскороем выражение согласно законам реляционной алгебры.

$$\pi_{e_1, d_1}(\sigma_{\varphi \wedge \Phi}(R \bowtie_{\theta} S)) = \pi_{e_1, d_1}(\sigma_{\Phi}(\sigma_{\varphi}(R \bowtie_{\theta} S)))$$

Т.к φ относится к атрибутам R , то справедливо:

$$\pi_{e_1, d_1}(\sigma_{\Phi}(\sigma_{\varphi}(R \bowtie_{\theta} S))) = \pi_{e_1, d_1}(\sigma_{\Phi}(\sigma_{\varphi}(R) \bowtie_{\theta} S)) = \pi_{e_1, d_1}(\sigma_{\Phi}(S \bowtie_{\theta} \sigma_{\varphi}(R)))$$

Т.к Φ относится к атрибутам S , то справедливо:

$$\pi_{e_1, d_1}(\sigma_{\Phi}(S \bowtie_{\theta} \sigma_{\varphi}(R))) = \pi_{e_1, d_1}(\sigma_{\Phi}(S) \bowtie_{\theta} \sigma_{\varphi}(R)) = \pi_{e_1, d_1}(\sigma_{\varphi}(R) \bowtie_{\theta} \sigma_{\Phi}(S))$$

Это соответствует второму плану. ■

ВЫБОР ОПТИМАЛЬНОГО ПЛАНА ВЫПОЛНЕНИЯ ЗАПРОСА

Наиболее оптимальным является **второй план**, потому что фильтрация выполняется раньше соединения, и уменьшает объем данных, который нужно соединять, что снижает затраты на операцию соединения.

ИНДЕКСЫ

```
-- 1)
create index "Н_ТИПЫ_ВЕДОМОСТЕЙ_ИД" on "Н_ТИПЫ_ВЕДОМОСТЕЙ" using
hash("ИД");
-- 2)
create index "Н_ВЕДОМОСТИ_ТВ_ИД" on "Н_ВЕДОМОСТИ" using
hash("ТВ_ИД");
-- 3)
create index "Н_ТИПЫ_ВЕДОМОСТЕЙ_НАИМЕНОВАНИЕ" on
"Н_ТИПЫ_ВЕДОМОСТЕЙ" using btree("НАИМЕНОВАНИЕ");
-- 4)
create index "Н_ВЕДОМОСТИ_ДАТА" on "Н_ВЕДОМОСТИ" using
btree("ДАТА");
```

- Первые два индекса являются hash-индексами, так как атрибуты “Н_ТИПЫ_ВЕДОМОСТЕЙ”.“ТВ_ИД” используются для конструкции join, где соединение происходит по равенству двух этих атрибутов, а hash-индексы очень эффективны именно в случае отношения строгого равенства.
- Вторые два индекса являются btree-индексами, так как атрибуты “Н_ТИПЫ_ВЕДОМОСТЕЙ”.“НАИМЕНОВАНИЕ” и “Н_ВЕДОМОСТИ”.“ДАТА” сравниваются с другими значениями в ходе запроса, btree индексы эффективно обрабатывают такие ситуации.
- Планы выполнения запросов изменятся, так как будет происходить индексный скан и nested loop join выполнится быстрее так как не будет выполняться полное сканирование внутренней таблицы .

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ EXPLAIN ANALYZE

```
Nested Loop  (cost=0.29..8.24 rows=1 width=422) (actual
time=0.027..0.028 rows=0 loops=1)
  Join Filter: ("Н_ТИПЫ_ВЕДОМОСТЕЙ"."ИД" =
"Н_ВЕДОМОСТИ"."ТВ_ИД")
    -> Seq Scan on "Н_ТИПЫ_ВЕДОМОСТЕЙ"  (cost=0.00..1.04 rows=1
```

```

width=422) (actual time=0.019..0.021 rows=1 loops=1)
    Filter: (("НАИМЕНОВАНИЕ")::text < 'Перезачет'::text)
    Rows Removed by Filter: 2
-> Index Scan using "ВЕД_ДАТА_I" on
    "Н_ВЕДОМОСТИ" (cost=0.29..7.19 rows=1 width=8) (actual
time=0.004..0.004 rows=0 loops=1)
    Index Cond: ("ДАТА" < '1998-01-05 00:00:00'::timestamp
without time zone)"
Planning Time: 0.264 ms
Execution Time: 0.062 ms

```

ЗАПРОС №2

ОПИСАНИЕ ЗАПРОСА

Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_ОБУЧЕНИЯ, Н_УЧЕНИКИ.

Вывести атрибуты: Н_ЛЮДИ.ИД, Н_ОБУЧЕНИЯ.НЗК, Н_УЧЕНИКИ.ИД.

Фильтры: (AND)

- а) Н_ЛЮДИ.ИМЯ < Александр.
- б) Н_ОБУЧЕНИЯ.НЗК < 999080.
- в) Н_УЧЕНИКИ.ГРУППА > 3100.

Вид соединения: RIGHT JOIN.

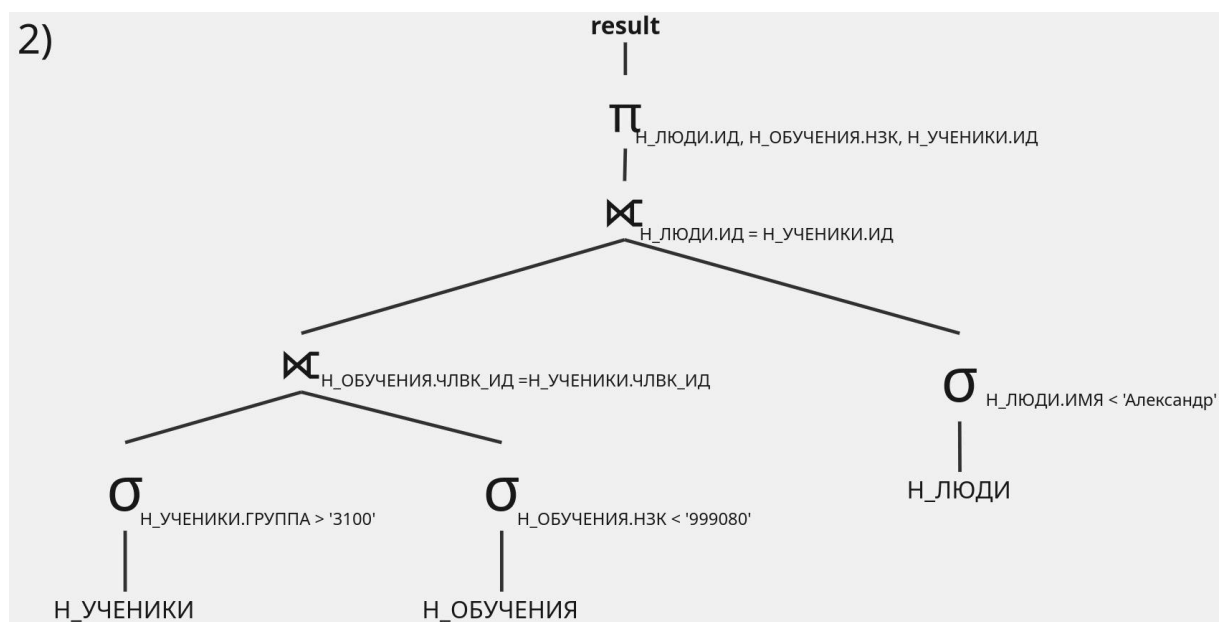
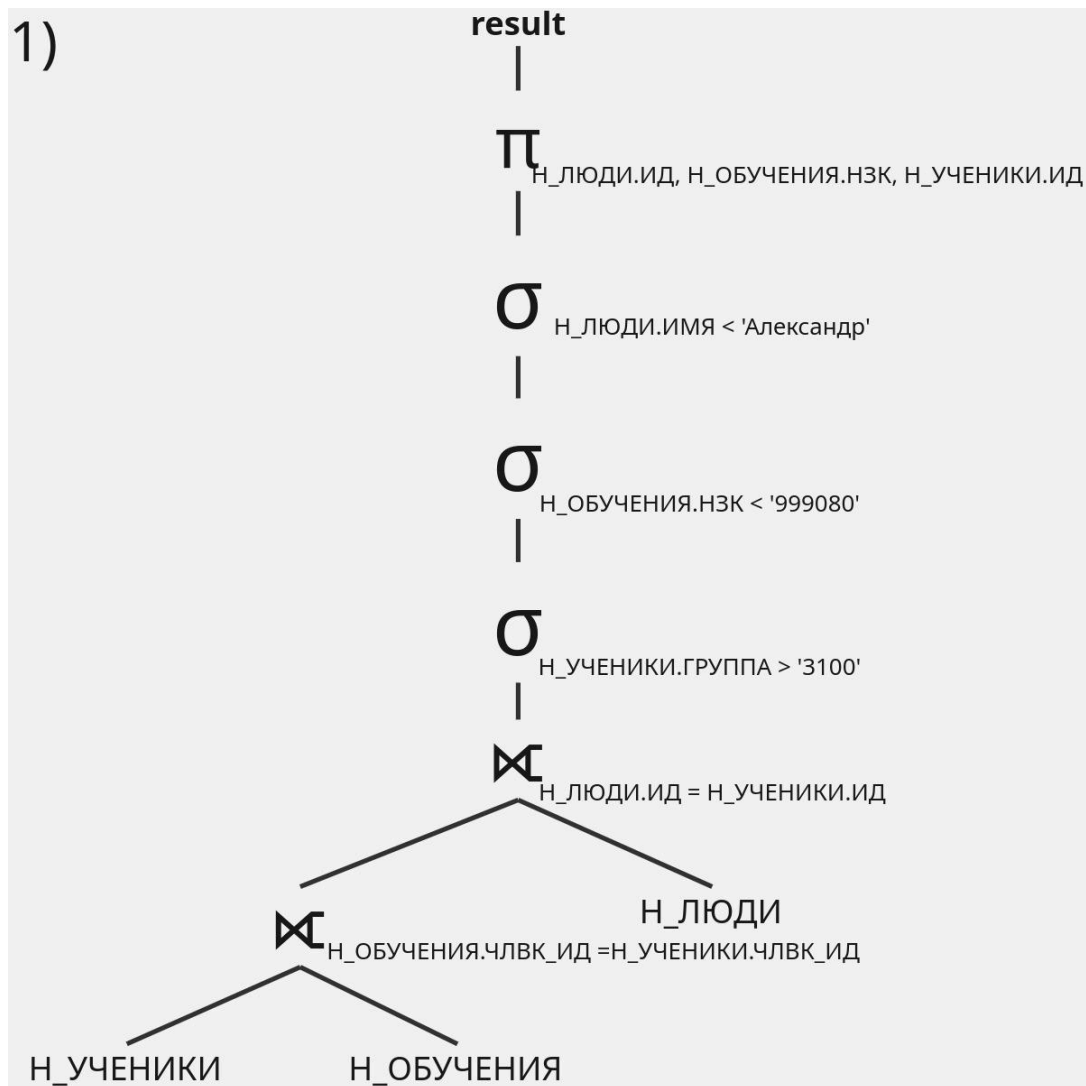
РЕАЛИЗАЦИЯ ЗАПРОСА НА SQL

```

select "Н_ЛЮДИ"."ИД", НО."НЗК", НУ."ИД" from "Н_ЛЮДИ"
right join "Н_УЧЕНИКИ" НУ on "Н_ЛЮДИ"."ИД" = НУ."ИД"
right join "Н_ОБУЧЕНИЯ" НО on НУ."ЧЛВК_ИД" = НО."ЧЛВК_ИД"
where "Н_ЛЮДИ"."ИМЯ" < 'Александр' and НО."НЗК" < '999080' and
НУ."ГРУППА" > '3100'

```

ПЛАНЫ ВЫПОЛНЕНИЯ ЗАПРОСА



ОТВЕТЫ НА ВОПРОСЫ, ПРЕДСТАВЛЕННЫЕ В ЗАДАНИИ

ВЫБОР ОПТИМАЛЬНОГО ПЛАНА ВЫПОЛНЕНИЯ ЗАПРОСА

Наиболее оптимальным является **второй план**, потому что фильтрация выполняется раньше соединения, и уменьшает объем данных, который нужно соединять, что снижает затраты на операцию соединения.

ИНДЕКСЫ

```
-- 1)
create index "Н_ЛЮДИ_ИД" on "Н_ЛЮДИ" using hash("ИД");
-- 2)
create index "Н_ЛЮДИ_ИМЯ" on "Н_ЛЮДИ" using btree("ИМЯ");
-- 3)
create index "Н_УЧЕНИКИ_ИД" on "Н_УЧЕНИКИ" using hash("ИД");
-- 4)
create index "Н_УЧЕНИКИ_ЧЛВК_ИД" on "Н_УЧЕНИКИ" using
hash("ЧЛВК_ИД");
-- 5)
create index "Н_УЧЕНИКИ_ГРУППА" on "Н_УЧЕНИКИ" using
btree("ГРУППА");
-- 6)
create index "Н_ОБУЧЕНИЯ_ЧЛВК_ИД" on "Н_ОБУЧЕНИЯ" using
hash("ЧЛВК_ИД");
-- 7)
create index "Н_ОБУЧЕНИЯ_НЗК" on "Н_ОБУЧЕНИЯ" using btree("НЗК");
```

- Часть индексов являются hash-индексами, так как атрибуты “Н_ЛЮДИ”.“ИД”, НУ.“ИД”, НУ.“ЧЛВК_ИД” и НО.“ЧЛВК_ИД” используются в конструкции JOIN, где соединение происходит по строгому равенству этих атрибутов. .
- Часть индексов являются btree-индексами, так как атрибуты “Н_ЛЮДИ”.“ИМЯ”, НО.“НЗК” и НУ.“ГРУППА” сравниваются с другими значениями с использованием операторов < и >. Btree-индексы эффективно поддерживают такие операции.
- Планы выполнения запросов изменятся, так как будет происходить индексный скан и hash join выполнится быстрее.

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ EXPLAIN ANALYZE

```
Nested Loop (cost=166.63..1104.17 rows=95 width=14) (actual
time=4.204..12.368 rows=5 loops=1)
  -> Hash Join (cost=166.35..1068.36 rows=107 width=12) (actual
time=4.189..12.328 rows=6 loops=1)
    "      Hash Cond: ("НУ"."ИД" = "Н_ЛЮДИ"."ИД")"
    "      -> Seq Scan on "Н_УЧЕНИКИ" "НУ" (cost=0.00..867.39
rows=13186 width=8) (actual time=0.015..8.246 rows=13206 loops=1)"
    "      Filter: (("ГРУППА")::text > '3100'::text)"
    Rows Removed by Filter: 10105
  -> Hash (cost=163.97..163.97 rows=190 width=4) (actual
time=2.697..2.698 rows=191 loops=1)
    Buckets: 1024 Batches: 1 Memory Usage: 15kB
    "      -> Seq Scan on "Н_ЛЮДИ" (cost=0.00..163.97
rows=190 width=4) (actual time=0.005..2.654 rows=191 loops=1)"
    "      Filter: (("ИМЯ")::text < 'Александр'::text)"
```

ВЫВОДЫ ПО РАБОТЕ

В ходе выполнения лабораторной работы я познакомился с индексами, оптимизацией запросов и выбором плана выполнения запросов, начал шире мыслить о запросах, которые я пишу и, об их эффективности с точки зрения работы СУБД. Подробнее познакомился с логикой реляционной алгебры, на которой завязана работа СУБД.